Bhai, mubarak ho! Part 1 (Rule-Based) aur Part 2 (Basic NLP with TF-IDF) complete kar liya. Ab **Part 3: Advanced AI** mein hum ise next level pe le jaate hain.

Project synopsis ke hisaab se focus hai **NLP techniques + intent classification** pe, aur TensorFlow/PyTorch optional hai. Toh hum **TensorFlow/Keras** use karenge (beginner-friendly, powerful, aur doc mein mentioned hai). Yeh deep learning model banayenge jo intents ko accurately classify karega – TF-IDF se better accuracy milegi, especially complex sentences mein (e.g., "fees kitni lagti hai semester ke liye?" ko sahi se pakdega).

Key Upgrades in Part 3:

- Neural Network model (Keras Sequential) for intent classification.
- Train model on DB se intents + patterns.
- Model save/load in Flask (har baar train na karna pade).
- Random response per intent (variety ke liye).
- Better preprocessing (stopwords remove, lemmatize).
- Confidence score dikhao chat mein.
- Admin panel mein multiple responses add/edit karne ka option.
- Same Flask + MySQL continue.
- Jupyter Notebook use karenge model training ke liye (install karna padega).

Tech Updates:

- New libraries: tensorflow, keras, joblib (model save ke liye)
- Jupyter install: `pip install notebook`
- Model file: `intent_model.h5` (save in project folder)

Part 3 Roadmap (5 Phases)

1. Setup + Libraries

2. Database Update for Multiple Responses

3. Jupyter Notebook mein Model Train Karo

4. Flask mein Model Load + Integrate in /chat

5. Admin Panel Upgrade + Testing

Phase 1: Setup + Libraries

Goal: TensorFlow install, Jupyter ready.

Commands (venv activate karo):

```
pip install tensorflow keras joblib jupyter
pip freeze > requirements.txt
```

Jupyter start karo:

```
jupyter notebook
```

→ Browser mein open hoga. New notebook banao naam `train_intent_model.ipynb`.

Common mistakes:

- TensorFlow slow? → CPU pe chalega (GPU nahi hai to).
- Import error? → pip install sahi se.

Next Checklist:

☑ Jupyter chal raha?

☑ Notebook bana?

Phase 2: Database Update – Multiple Responses

Goal: Ek intent ke multiple responses (random choose karne ke liye).

New Table (MySQL mein run):

```sql
CREATE TABLE responses (
   id INT AUTO_INCREMENT PRIMARY KEY,
   intent_id INT NOT NULL,
   response_text TEXT NOT NULL,
   FOREIGN KEY (intent_id) REFERENCES intents(id) ON DELETE CASCADE
);
```

Migrate old data (purana intents.response ko responses mein daalo):

```sql
INSERT INTO responses (intent_id, response_text)
SELECT id, response FROM intents;
```

Ab intents table se response column drop kar sakte ho (optional):

```sql
ALTER TABLE intents DROP COLUMN response;
```

Sample (ek intent ke multiple responses):

```sql
-- Assume greeting intent_id=1
INSERT INTO responses (intent_id, response_text) VALUES
(1, 'Hello! Kaise ho? 😊 '),
(1, 'Hi there! Kya chal raha hai?'),
(1, 'Namaste! Help chahiye?');
```

Next Checklist:

☑ responses table bana?

☑ Multiple entries per intent?

Phase 3: Jupyter Notebook mein Model Train Karo

Goal: Keras model train + save.

train_intent_model.ipynb mein copy-paste (step-by-step cells):

Cell 1: Imports

```python
import mysql.connector
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.utils import to_categorical
import joblib
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('stopwords')
```

Cell 2: DB Connect + Data Load

```python
def get_db_connection():
    return mysql.connector.connect(
        host='localhost',
        user='root',
        password='your_password',
        database='chatbot_db'
    )
conn = get_db_connection()
cursor = conn.cursor(dictionary=True)

# Load intents & patterns
cursor.execute("SELECT id, tag FROM intents")
intents = cursor.fetchall()
# Load patterns
patterns = []
labels = []
for intent in intents:
    cursor.execute("SELECT pattern FROM patterns WHERE intent_id = %s", (intent['id'],))
    intent_patterns = cursor.fetchall()
    for p in intent_patterns:
        patterns.append(p['pattern'])
        labels.append(intent['tag'])
cursor.close()
conn.close()
print(f"Loaded {len(patterns)} patterns")
```

Cell 3: Preprocessing (better than before)

```python
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    tokens = nltk.word_tokenize(text)
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word.isalnum() and word not in stop_words]
    return ' '.join(tokens)

cleaned_patterns = [clean_text(p) for p in patterns]
```

Cell 4: TF-IDF + Encoding

```python
vectorizer = TfidfVectorizer(max_features=500)  # Adjust max_features
X = vectorizer.fit_transform(cleaned_patterns).toarray()

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(labels)
y_cat = to_categorical(y)  # One-hot

joblib.dump(vectorizer, 'vectorizer.pkl')
joblib.dump(label_encoder, 'label_encoder.pkl')
print("Vectorizer & Encoder saved!")
```

Cell 5: Build & Train Model

```python
model = Sequential()
model.add(Dense(128, input_shape=(X.shape[1],), activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(y_cat.shape[1], activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y_cat, epochs=50, batch_size=8, validation_split=0.2)

model.save('intent_model.h5')
print("Model trained and saved!")
```

Cell 6: Test Prediction (optional)

```python
def predict_intent(message):
    clean = clean_text(message)
    vec = vectorizer.transform([clean]).toarray()
    pred = model.predict(vec)
    intent_idx = np.argmax(pred)
    confidence = np.max(pred)
    return label_encoder.inverse_transform([intent_idx])[0], confidence

print(predict_intent("hi there"))  # Should give 'greeting' high confidence
```

Run all cells (Shift+Enter).

Model `intent_model.h5`, `vectorizer.pkl`, `label_encoder.pkl` folder mein save honge.

Common mistakes:

- Data kam? → Zyada patterns daalo DB mein.
- Accuracy low? → Epochs badhao (100), ya layers add.
- GPU nahi? → Normal chalega.

Next Checklist:

☑ Model save hua?

☑ Test prediction sahi?

Phase 4: Flask mein Model Load + Integrate

Goal: /chat mein DL model use.

app.py mein add (top imports):

```python
from tensorflow.keras.models import load_model
import joblib
import numpy as np

# Load once
model = load_model('intent_model.h5')
vectorizer = joblib.load('vectorizer.pkl')
label_encoder = joblib.load('label_encoder.pkl')
```

Replace /chat route (poora):

```python
@app.route('/chat', methods=['POST'])
def chat():
    data = request.json
    user_message = data.get('message', '').strip()
    if not user_message:
        return jsonify({"response": "Kuch type karo!"})
```

```
    # Preprocess same as training
    clean = clean_text(user_message)  # Use same clean_text function (copy from notebook)
    if not clean.strip():
        return jsonify({"response": "Sorry, samajh nahi aaya."})

    vec = vectorizer.transform([clean]).toarray()
    pred = model.predict(vec, verbose=0)
    intent_idx = np.argmax(pred)
    confidence = float(np.max(pred))  # To show

    if confidence > 0.6:  # Higher threshold for DL
        predicted_tag = label_encoder.inverse_transform([intent_idx])[0]

        # Get random response from responses table
        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)
        cursor.execute("SELECT response_text FROM responses WHERE intent_id = (SELECT id
FROM intents WHERE tag = %s) ORDER BY RAND() LIMIT 1", (predicted_tag,))
        resp = cursor.fetchone()
        response = resp['response_text'] if resp else "Got it!"
        cursor.close()
        conn.close()
    else:
        # Fallback
        fallback_resp = "Sorry bhai, thoda clear bolo. Try: hi, fees, bye"
        response = fallback_resp
```

```
    # Log
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("INSERT INTO chat_logs (user_message, bot_response) VALUES (%s, %s)", (user_message, response))
        conn.commit()
    except:
        pass
    finally:
        cursor.close()
        conn.close()

    return jsonify({"response": response, "confidence": round(confidence * 100, 2)})
```

Update clean_text function in app.py (notebook se copy):

```python
# Same as notebook
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    tokens = nltk.word_tokenize(text)
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word.isalnum() and word not in stop_words]
    return ' '.join(tokens)
```

Chat UI mein confidence dikhao (chat.js update optional):

- addMessage(data.response + ` (Confidence: ${data.confidence}%)`, false);

Next Checklist:

☑ /chat POST se response + confidence?

☑ Random response har baar alag?

Phase 5: Admin Panel Upgrade + Testing

Goal: Admin mein responses manage.

New Routes (app.py):

- GET /admin/responses/<intent_id> → List responses

- POST /admin/responses → Add (intent_id, response_text)

- DELETE /admin/responses/<id>

Simple JS add in admin.html (similar to patterns):

- New section: Select intent → Add response textarea → Add button.

- Table show responses per intent.

Testing Checklist:

☑ Complex query test: "bhai fees kitni hai ek semester ki?" → fees intent high confidence.

☑ "shukriya" → thanks intent.

☑ Confidence <60% → fallback.

☑ Multiple responses random?

☑ Model retrain: New patterns/responses daalo → Jupyter rerun → files replace → app restart.

Final Recap Part 3:

- Advanced AI: Deep Learning (Keras NN) for intent classification.

- Better accuracy than TF-IDF.

- Random responses, confidence score.

- Matches synopsis: Model Design (NLP + intent classification), Chatbot Development (integrate model).

- Future: LSTM/RNN for sequences, BERT (advanced), voice (speech_recognition).

Bhai, Part 3 ready! 🚀

Jupyter mein train kar ke files project folder mein copy kar lo. Koi error ya accuracy issue ho to bata – fix karte hain.

Ab full project test kar, report bana, ya presentation ke liye slides chahiye to bol!